COIL

Cognitive Orchestration Interface Language

Операторский язык сценариев для когнитивной оркестрации

Учебник

Черновик 0.1 — 21 марта 2026

Этот учебник проведёт вас от первого протокола до трёх самых востребованных сценариев. По пути вы узнаете, какие решения принимает язык за вас, а какие остаются за вами — и как избежать типичных ошибок.

Contents

1 С чего начинать **2**
1.1 Что такое протокол COIL . . . 2
1.2 Минимальный протокол . . . 2
1.2.1 Два главных сигила . . . 3
1.3 Добавляем контекст: роль и входные данные . . . 3
1.3.1 Порядок модификаторов в ДУМАЙ . . . 4
1.4 Все сигилы . . . 4
1.5 Чек-лист: первый протокол . . . 4

2 Три частых сценария **4**
2.1 Классификация и маршрутизация . . . 5
2.1.1 Что здесь важно . . . 6
2.2 Параллельная экспертиза . . . 6
2.2.1 Что здесь важно . . . 8
2.3 Итеративное улучшение . . . 8
2.3.1 Что здесь важно . . . 10

3 ДУМАЙ или НАПИШИ с ЖДАТЬ? **10**
3.1 Два способа получить ответ . . . 10
3.2 Правило выбора . . . 10
3.3 НАПИШИ без ЖДАТЬ . . . 11
3.4 Три политики ожидания . . . 11

4 Как не делать **12**
4.1 Всё в одном ДУМАЙ . . . 12
4.2 ДУМАЙ для детерминированного решения . . . 12
4.3 Забыть ЖДИ перед использованием результата . . . 13
4.4 Повторное ОПРЕДЕЛИ . . . 13
4.5 Цикл без лимита . . . 14

5 Типичные ошибки **14**
5.1 Порядок модификаторов в ДУМАЙ . . . 14
5.2 РЕЗУЛЬТАТ не последний . . . 14
5.3 Шаблон << >> в ВЫПОЛНИ . . . 15

5.4 Необъявленные участники и инструменты 15
5.5 Путаница $name и ?name 16

6 Что дальше 16
6.1 Extended-операторы 16
6.2 Инструменты и ВЫПОЛНИ 16
6.3 Потоки и события 17
6.4 Где искать примеры 17

С чего начинать

Что такое протокол COIL

Протокол COIL — это сценарий для агента: что обдумать, кого спросить, когда подождать, когда ответить. Он описывает один конкретный случай работы, а не «всё поведение» агента навсегда.

Протокол читается сверху вниз, как чек-лист. Каждая строка — явное действие: подумать, отправить сообщение, дождаться ответа, завершить. Никакой скрытой логики внутри длинного промпта.

Минимальный протокол

Самый короткий рабочий протокол состоит из четырёх шагов: подумать, подождать, написать, выйти.

```
ДУМАЙ приветствие
  ЦЕЛЬ <<
  Придумай короткое приветственное сообщение.
  >>
  РЕЗУЛЬТАТ
  * текст: ТЕКСТ - приветствие
КОНЕЦ

ЖДИ
  НА ?приветствие
КОНЕЦ

НАПИШИ
  КОМУ @пользователь
  <<
  $приветствие.текст
  >>
КОНЕЦ

ВЫХОД
```

Разберём по шагам:

1. `ДУМАЙ приветствие` — запускает когнитивный шаг. LLM получает задание и начинает думать. Оператор не ждёт результата — он сразу создаёт **обещание** `?приветствие` и продолжает.

2. `ЖДИ НА ?приветствие` — точка синхронизации. Протокол останавливается, пока LLM не закончит. После этого результат доступен как **значение** `$приветствие`.

3. `НАПИШИ КОМУ @пользователь` — отправляет сообщение. Внутри шаблона `<< >>` подставляется `$приветствие.текст` — поле из структурированного результата.

4. `ВЫХОД` — завершает протокол.

Два главных сигила

- `$имя` — **значение**. Данные, которые уже доступны.
- `?имя` — **обещание**. Результат, который ещё не готов.

Правило простое: запускающий оператор (`ДУМАЙ`, `ВЫПОЛНИ`, `НАПИШИ`) создаёт `?имя`. После `ЖДИ` обещание разрешается в `$имя`.

Добавляем контекст: роль и входные данные

Минимальный протокол работает, но ему не хватает настройки. Добавим роль для LLM и входные данные от пользователя.

```
ОПРЕДЕЛИ аналитик
<<
Ты опытный аналитик данных. Отвечаешь коротко и по делу.
>>
КОНЕЦ

ПОЛУЧИ запрос
<<
Вопрос пользователя для анализа.
>>
КОНЕЦ

ДУМАЙ ответ
  КАК $аналитик
  ЦЕЛЬ <<
  Ответь на вопрос пользователя.
  >>
  ВХОД <<
  $запрос
  >>
  РЕЗУЛЬТАТ
  * ответ: ТЕКСТ - ответ на вопрос
КОНЕЦ

ЖДИ
  НА ?ответ
КОНЕЦ

НАПИШИ
  КОМУ @пользователь
  <<
  $ответ.ответ
  >>
КОНЕЦ

ВЫХОД
```

Новые элементы:

- `ОПРЕДЕЛИ аналитик` — создаёт именованное значение. Это мгновенный оператор: он не вызывает LLM, а просто сохраняет текст под именем `$аналитик`.

- `ПОЛУЧИ запрос` — блокирующий оператор. Протокол останавливается, пока среда не предоставит значение. Это может быть сообщение пользователя, параметр из конфигурации — протоколу всё равно, откуда.
- `КАК $аналитик` — модификатор-оснащение в `ДУМАЙ`. Говорит LLM, в какой роли действовать.
- `ВХОД << ... >>` — модификатор-постановка. Условие задачи.

Порядок модификаторов в ДУМАЙ

Модификаторы `ДУМАЙ` делятся на две группы, и группы не смешиваются:

1. **Оснащение** (чем и как): `ЧЕРЕЗ`, `КАК`, `ИСПОЛЬЗУЯ`
2. **Постановка** (что решать): `ЦЕЛЬ`, `ВХОД`, `КОНТЕКСТ`, `РЕЗУЛЬТАТ`

Оснащение **всегда перед** постановкой. `РЕЗУЛЬТАТ` **всегда последний**.

Все сигилы

В предыдущих примерах вы видели `$` и `?`. Полный набор:

Сигил	Значение	Пример
`$`	Значение	`$запрос`, `$план.summary`
`?`	Обещание	`?план`, `?ответ`
`@`	Участник	`@тех_аналитик`, `@пользователь`
`!`	Инструмент	`!загрузить_статью`
`#`	Адрес (канал)	`#быстрые_вопросы`
`~`	Поток	`~решение`

Участники и инструменты требуют предварительного объявления через `УЧАСТНИКИ` и `ИНСТРУМЕНТЫ`.

Чек-лист: первый протокол

Минимум для работающего протокола:

- Хотя бы один когнитивный шаг (`ДУМАЙ`) или действие (`ВЫПОЛНИ`, `НАПИШИ`).
- `ЖДИ` перед использованием результата.
- Явное завершение (`ВЫХОД`).

Рекомендуется:

- `УЧАСТНИКИ` — кто участвует в протоколе.
- `ИНСТРУМЕНТЫ` — какие инструменты доступны.
- `ОПРЕДЕЛИ` — роли и константы.

Три частых сценария

Большинство агентных задач укладываются в три паттерна. Освойте их — и вы закроете 80% случаев.

Классификация и маршрутизация

Задача: входящий запрос нужно классифицировать и направить подходящему специалисту.

```
ОПРЕДЕЛИ general_role
<<
Ты специалист по обслуживанию клиентов.
>>
КОНЕЦ

ОПРЕДЕЛИ technical_role
<<
Ты специалист технической поддержки.
Решаешь проблемы пошагово.
>>
КОНЕЦ

' --- Классификация ---

ДУМАЙ classification
  ЦЕЛЬ <<
  Классифицируй запрос клиента.
  >>
  ВХОД <<
  $message
  >>
  РЕЗУЛЬТАТ
  * type: ВЫБОР(general, technical) - тип запроса
КОНЕЦ

ЖДИ
  НА ?classification
КОНЕЦ

' --- Выбор роли ---

ОПРЕДЕЛИ current_role
$general_role
КОНЕЦ

ЕСЛИ $classification.type == "technical"
  УСТАНОВИ $current_role
  $technical_role
  КОНЕЦ
КОНЕЦ

' --- Ответ от выбранного специалиста ---

ДУМАЙ response
  КАК $current_role
  ВХОД <<
  $message
  >>
  РЕЗУЛЬТАТ
  * answer: ТЕКСТ - ответ клиенту
```

```
КОНЕЦ

ЖДИ
  НА ?response
КОНЕЦ

НАПИШИ
  КОМУ @пользователь
  <<
  $response.answer
  >>
КОНЕЦ

ВЫХОД
```

Что здесь важно

- `ВЫБОР(general, technical)` в `РЕЗУЛЬТАТ` — ограничивает LLM фиксированным набором значений. Именно это делает ветвление надёжным: `ЕСЛИ` получает гарантированно корректное значение.
- `ЕСЛИ` — вычисляется средой исполнения **детерминированно**, без вызова LLM. Это управляющая логика, а не когнитивная.
- Паттерн: **сначала думаем, чтобы классифицировать** — затем ветвимся детерминированно по результату.

Параллельная экспертиза

Задача: два эксперта параллельно анализируют код, техлид агрегирует результаты.

```
ОПРЕДЕЛИ security_role
<<
Ты эксперт по безопасности кода.
>>
КОНЕЦ

ОПРЕДЕЛИ performance_role
<<
Ты эксперт по производительности кода.
>>
КОНЕЦ

ОПРЕДЕЛИ tech_lead_role
<<
Ты технический руководитель.
Подводишь итоги нескольких проверок.
>>
КОНЕЦ

' --- Два ревью запускаются подряд ---
' --- Оба работают параллельно ---
```

```
ДУМАЙ security_review
  КАК $security_role
  ВХОД <<
  $message
  >>
  РЕЗУЛЬТАТ
  * issues: ТЕКСТ - найденные уязвимости
  * risk: ВЫБОР(low, medium, high) - уровень риска
КОНЕЦ

ДУМАЙ performance_review
  КАК $performance_role
  ВХОД <<
  $message
  >>
  РЕЗУЛЬТАТ
  * issues: ТЕКСТ - найденные проблемы
  * impact: ВЫБОР(low, medium, high) - влияние
КОНЕЦ

' --- Ждём оба результата ---

ЖДИ
  НА ?security_review, ?performance_review
  РЕЖИМ ВСЕ
КОНЕЦ

' --- Агрегация ---

ДУМАЙ summary
  КАК $tech_lead_role
  ЦЕЛЬ <<
  Объедини результаты двух ревью в итог.
  >>
  КОНТЕКСТ <<
  Безопасность: $security_review
  Производительность: $performance_review
  >>
  РЕЗУЛЬТАТ
  * summary: ТЕКСТ - итоговое резюме
  * actions: ТЕКСТ - ключевые действия
КОНЕЦ

ЖДИ
  НА ?summary
КОНЕЦ

НАПИШИ
  КОМУ @пользователь
  <<
  $summary.summary

  Действия: $summary.actions
  >>
КОНЕЦ
```

```
ВЫХОД
```

Что здесь важно

- **Запускающие операторы не блокируют.** `ДУМАЙ security_review` создаёт обещание и тут же переходит к следующему `ДУМАЙ performance_review`. Оба работают параллельно.
- `ЖДИ НА ?security_review, ?performance_review РЕЖИМ ВСЕ` — явная точка синхронизации. Протокол ждёт, пока **оба** результата не будут готовы.
- `РЕЖИМ ЛЮБОЙ` — альтернатива: продолжить, как только готов хотя бы один результат.
- Паттерн: **запустить несколько**, потом **подождать все**, потом **агрегировать**.

Итеративное улучшение

Задача: перевод текста с циклом проверки качества. Оценщик проверяет перевод, и если качество недостаточно — переводчик улучшает.

```
ОПРЕДЕЛИ translator_role
<<
Ты эксперт по литературному переводу.
>>
КОНЕЦ

ОПРЕДЕЛИ evaluator_role
<<
Ты эксперт по оценке литературных переводов.
>>
КОНЕЦ

' --- Первый перевод ---

ДУМАЙ initial_translation
  КАК $translator_role
  ЦЕЛЬ <<
  Переведи текст, сохраняя тон и нюансы.
  >>
  ВХОД <<
  $message
  >>
  РЕЗУЛЬТАТ
  * text: ТЕКСТ - перевод
КОНЕЦ

ЖДИ
  НА ?initial_translation
КОНЕЦ

ОПРЕДЕЛИ current
$initial_translation.text
КОНЕЦ
```

```
ОПРЕДЕЛИ done
0
КОНЕЦ

' --- Цикл оценки и улучшения ---

ПОВТОРЯЙ ДО $done НЕ БОЛЕЕ 3

  ДУМАЙ evaluation
    КАК $evaluator_role
    ЦЕЛЬ <<
    Оцени качество перевода.
    >>
    ВХОД <<
    Оригинал: $message
    Перевод: $current
    >>
    РЕЗУЛЬТАТ
    * score: ЧИСЛО - качество от 1 до 10
    * issues: ТЕКСТ - проблемы
  КОНЕЦ

  ЖДИ
    НА ?evaluation
  КОНЕЦ

  ЕСЛИ $evaluation.score >= 8
    УСТАНОВИ $done
    1
    КОНЕЦ
  КОНЕЦ

  ЕСЛИ $done == 0
    ДУМАЙ improved
      КАК $translator_role
      ЦЕЛЬ <<
      Улучши перевод на основе замечаний.
      >>
      ВХОД <<
      Оригинал: $message
      Перевод: $current
      >>
      КОНТЕКСТ <<
      Проблемы: $evaluation.issues
      >>
      РЕЗУЛЬТАТ
      * text: ТЕКСТ - улучшенный перевод
    КОНЕЦ

    ЖДИ
      НА ?improved
    КОНЕЦ

    УСТАНОВИ $current
    $improved.text
```

```
    КОНЕЦ
  КОНЕЦ

КОНЕЦ

НАПИШИ
  КОМУ @пользователь
  <<
  $current
  >>
КОНЕЦ

ВЫХОД
```

Что здесь важно

- `ПОВТОРЯЙ ДО $done НЕ БОЛЕЕ 3` — цикл с обязательным лимитом. Это не стилистическое требование, а часть конструкции: каждая итерация с `ДУМАЙ` расходует бюджет.
- `УСТАНОВИ` меняет существующее значение (а не `ОПРЕДЕЛИ`, который создаёт новое).
- Паттерн: **работник создаёт**, **оценщик проверяет**, **цикл повторяется до нужного качества**.

ДУМАЙ или НАПИШИ с ЖДАТЬ?

Это центральный выбор при проектировании протокола. Оба способа дают результат — но природа результата разная.

Два способа получить ответ

	ДУМАЙ	**НАПИШИ с ЖДАТЬ**
Кто работает	Текущий агент (LLM)	Другой участник
Результат определяется	Структурой `РЕЗУЛЬТАТ`	Ответом участника
Контроль формата	Полный (типы, поля)	Зависит от участника
Когда использовать	Нужна когнитивная работа этого агента	Нужен ответ от другого агента или человека

Правило выбора

- **Агент справится сам?** → `ДУМАЙ`
- **Нужен ответ от другого?** → `НАПИШИ` с `ЖДАТЬ`

Примеры:

Задача	Оператор
Классифицировать запрос	ДУМАЙ
Спросить у техника диагноз	НАПИШИ ЖДАТЬ ЛЮБОЙ
Собрать выжимку из данных	ДУМАЙ
Уточнить у пользователя детали	НАПИШИ ЖДАТЬ ЛЮБОЙ
Попросить троих экспертов	НАПИШИ ЖДАТЬ ВСЕ

НАПИШИ без ЖДАТЬ

Если ЖДАТЬ не указан и у НАПИШИ нет имени результата — это отправка без ожидания: сообщение ушло, протокол продолжается.

Используйте для: финального ответа, уведомлений, статусных сообщений.

```
' Финальный ответ --- не ждём
НАПИШИ
  КОМУ @пользователь
  <<
  Всё готово: $result.summary
  >>
КОНЕЦ
```

Три политики ожидания

Политика	Когда обещание разрешается
ЖДАТЬ НЕТ	Сообщение отправлено (аналог отсутствия ЖДАТЬ)
ЖДАТЬ ЛЮБОЙ	Получен хотя бы один ответ
ЖДАТЬ ВСЕ	Получены ответы от всех адресатов

Пример запроса с ожиданием:

```
НАПИШИ вопрос
  КОМУ @тех_аналитик
  ЖДАТЬ ЛЮБОЙ
  НЕ БОЛЕЕ 10м
  <<
  Проверь возможную причину сбоя.
  >>
КОНЕЦ

ЖДИ
  НА ?вопрос
КОНЕЦ

' Теперь $вопрос содержит ответ аналитика
```

Как не делать

Всё в одном ДУМАЙ

Соблазн: описать весь рабочий процесс в одном гигантском промпте.

```
' Плохо: вся логика спрятана в промпте
ДУМАЙ всё_сразу
  ЦЕЛЬ <<
  Классифицируй запрос, выбери специалиста,
  составь ответ, проверь качество, и если
  плохо --- переделай.
  >>
  РЕЗУЛЬТАТ
  * answer: ТЕКСТ
КОНЕЦ
```

Почему плохо:

- Логика ветвлений спрятана внутри промпта — теряется детерминизм.
- Невозможно проверить промежуточные результаты.
- Один шаг тратит весь бюджет.
- При ошибке на любом этапе — всё заново.

Правильно: разбить на отдельные шаги, соединённые управляющей логикой (как в сценарии «Классификация и маршрутизация»).

ДУМАЙ для детерминированного решения

```
' Плохо: LLM решает, больше ли score 80
ДУМАЙ check
  ЦЕЛЬ <<
  Определи, превышает ли оценка 80.
  >>
  ВХОД <<
  Оценка: $evaluation.score
  >>
  РЕЗУЛЬТАТ
  * exceeds: ФЛАГ
КОНЕЦ
```

Зачем тратить токены на то, что среда вычислит мгновенно?

```
' Правильно: детерминированная проверка
ЕСЛИ $evaluation.score >= 80
  ...
КОНЕЦ
```

ЕСЛИ вычисляется без LLM. Оставляйте ДУМАЙ для задач, где нужно **понимание**, а не арифметика.

Забыть ЖДИ перед использованием результата

```
' Плохо: $plan ещё не готов
ДУМАЙ plan
  ЦЕЛЬ <<
  Составь план.
  >>
  РЕЗУЛЬТАТ
  * steps: ТЕКСТ
КОНЕЦ

НАПИШИ
  КОМУ @пользователь
  <<
  План: $plan.steps
  >>
КОНЕЦ
```

После ДУМАЙ существует **обещание** `?plan`, но **значение** `$plan` ещё недоступно. Нужна явная синхронизация:

```
ДУМАЙ plan
  ...
КОНЕЦ

ЖДИ
  НА ?plan
КОНЕЦ

НАПИШИ
  КОМУ @пользователь
  <<
  План: $plan.steps
  >>
КОНЕЦ
```

Повторное ОПРЕДЕЛИ

```
' Плохо: ошибка подготовки
ОПРЕДЕЛИ counter
0
КОНЕЦ

ОПРЕДЕЛИ counter
1
КОНЕЦ
```

ОПРЕДЕЛИ создаёт **новое** имя. Повторное ОПРЕДЕЛИ с тем же именем — ошибка подготовки. Для изменения — УСТАНОВИ:

```
ОПРЕДЕЛИ counter
0
КОНЕЦ

УСТАНОВИ $counter
1
КОНЕЦ
```

Цикл без лимита

В COIL ПОВТОРЯЙ без лимита невозможен — это часть конструкции. Зачем?

Каждая итерация с ДУМАЙ внутри тратит токены. Без верхней границы протокол может исчерпать весь бюджет. Лимит — это не перестраховка, а осознанное проектное решение.

Типичные ошибки

Порядок модификаторов в ДУМАЙ

```
' Ошибка подготовки: ЦЕЛЬ (постановка) перед КАК (оснащение)
ДУМАЙ plan
  ЦЕЛЬ <<
  Найди проблему.
  >>
  КАК $analyst_role
КОНЕЦ
```

Правило: **оснащение** (ЧЕРЕЗ, КАК, ИСПОЛЬЗУЯ) **перед постановкой** (ЦЕЛЬ, ВХОД, КОНТЕКСТ, РЕЗУЛЬТАТ).

```
ДУМАЙ plan
  КАК $analyst_role
  ЦЕЛЬ <<
  Найди проблему.
  >>
КОНЕЦ
```

РЕЗУЛЬТАТ не последний

```
' Ошибка: КОНТЕКСТ после РЕЗУЛЬТАТ
ДУМАЙ analysis
  РЕЗУЛЬТАТ
  * summary: ТЕКСТ
  КОНТЕКСТ <<
```

```
  Дополнительные данные.
  >>
КОНЕЦ
```

РЕЗУЛЬТАТ **обязан** быть последним модификатором в ДУМАЙ.

Шаблон << >> в ВЫПОЛНИ

```
' Ошибка подготовки: шаблон запрещён в ВЫПОЛНИ
ВЫПОЛНИ article
  ИСПОЛЬЗУЯ !загрузить_статью
  <<
  Загрузи статью по адресу $url
  >>
КОНЕЦ
```

ВЫПОЛНИ — вызов инструмента. Инструменты принимают структурированные аргументы, а не свободный текст:

```
ВЫПОЛНИ article
  ИСПОЛЬЗУЯ !загрузить_статью
  - url: $url
  - format: "markdown"
КОНЕЦ
```

Шаблоны << >> предназначены для естественного языка (ДУМАЙ, НАПИШИ). Аргументы `- key: value` — для вызовов инструментов (ВЫПОЛНИ).

Необъявленные участники и инструменты

```
' Ошибка подготовки: @аналитик не объявлен
НАПИШИ
  КОМУ @аналитик
  <<
  Проверь данные.
  >>
КОНЕЦ
```

Все участники должны быть объявлены через УЧАСТНИКИ, все инструменты — через ИНСТРУМЕНТЫ:

```
УЧАСТНИКИ аналитик

НАПИШИ
  КОМУ @аналитик
  <<
  Проверь данные.
```

```
  >>
КОНЕЦ
```

Путаница $name и ?name

```
' Ошибка: ЖДИ ожидает обещание (?), а не значение ($)
ЖДИ
  НА $plan
КОНЕЦ
```

`ЖДИ НА` принимает **обещания** (`?name`). `$name` — это уже разрешённое значение, ждать его бессмысленно.

```
ЖДИ
  НА ?plan
КОНЕЦ
```

Что дальше

Extended-операторы

В примерах выше вы уже использовали `ЕСЛИ` и `ПОВТОРЯЙ`. Они относятся к Extended-набору, который также включает:

- `КАЖДЫЙ` — итерация по элементам списка.
- `СОБЕРИ` — агрегация набора результатов.
- `СИГНАЛ` — отправка follow-up в существующий поток.

Extended-операторы стабильны по идее, но их точная форма может уточняться.

Инструменты и ВЫПОЛНИ

`ВЫПОЛНИ` вызывает внешние инструменты: загрузку данных, API, интерактивные сессии. Инструмент создаёт и обещание `?name`, и поток `~name` (для интерактивных сессий).

```
ИНСТРУМЕНТЫ загрузить_статью

ВЫПОЛНИ article
  ИСПОЛЬЗУЯ !загрузить_статью
  - url: $url
КОНЕЦ
```

Потоки и события

Каждый запускающий оператор создаёт не только обещание (`?name`), но и поток (`~name`). Поток — это двусторонний канал для follow-up сообщений. Через `СИГНАЛ` можно дослать данные в работающий шаг:

```
СИГНАЛ ~решение
  ТИП УТОЧНЕНИЕ
  <<
  Клиент добавил новую информацию.
  >>
КОНЕЦ
```

Где искать примеры

- `examples/hello-world.md` — минимальный протокол
- `examples/patterns/routing.coil` — классификация и маршрутизация
- `examples/patterns/parallelization.coil` — параллельная экспертиза
- `examples/patterns/evaluator-optimizer.coil` — итеративное улучшение
- `examples/patterns/orchestrator-workers.coil` — планирование и параллельные воркеры
- `examples/patterns/prompt-chaining.coil` — цепочка промптов

Полная спецификация — в директории `spec/`.